

리눅스 3.4 - B+Tree

남궁명수

[B+Tree]

자료구조:

- B-Tree를 데이터베이스 인덱스에 적합하게 변형한 다진 균형 탐색 트리

내부 노드:

- 탐색 기준이 되는 키
- 자식 페이지를 가리키는 포인터
- 실제 레코드 정보는 일반적으로 저장하지 않음

리프 노드:

- 모든 인덱스 키
- 실제 레코드 또는 실제 레코드를 찾기 위한 정보
- 이전·다음 리프 노드와의 연결 정보

탐색:

- 내부 노드의 키를 비교하며 자식 페이지로 이동
- 최종적으로 리프 노드에서 키와 레코드 정보 확인

삽입:

- 리프 노드에 키 삽입
- 노드가 가득 차면 분할
- 새로운 경계 키를 부모 내부 노드에 복사

삭제:

- 리프 노드에서 키 삭제
- 키가 부족하면 형제에게 빌리거나 병합
- 필요하면 부모 노드까지 균형 조정

시간 복잡도:

- 탐색: $O(\log N)$
- 삽입: $O(\log N)$
- 삭제: $O(\log N)$

데이터베이스에 적합한 이유:

- 높은 Fan-out으로 트리 높이가 낮음
- 적은 페이지 I/O로 검색 가능

- 키가 정렬된 상태로 유지됨
- 리프 노드 연결을 통한 효율적인 범위 검색
- 등가 검색, 범위 검색, 정렬을 모두 지원

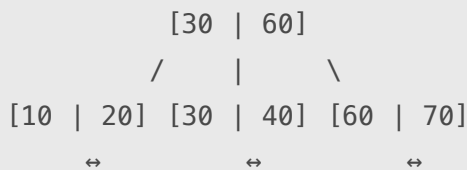
B+Tree는 B-Tree를 데이터베이스 인덱스와 파일 시스템에 더 적합하도록 변형한 다진 균형 탐색 트리이다.

B-Tree와 마찬가지로 다음 특징을 가진다.

- 노드 하나에 여러 개의 키를 저장한다.
- 노드 하나가 여러 개의 자식을 가질 수 있다.
- 키가 정렬된 상태로 유지된다.
- 모든 리프 노드의 깊이가 같다.
- 삽입과 삭제 후에도 트리의 균형을 유지한다.

B+Tree만의 가장 중요한 특징은 다음 두 가지다.

1. 실제 레코드를 찾기 위한 정보는 리프 노드에 모아 저장한다.
2. 리프 노드끼리 키 순서대로 연결되어 있다.



내부 노드는 탐색 방향을 결정하고, 리프 노드는 실제 검색 결과를 제공한다.

[1] [B+Tree의 구성]

B+Tree는 크게 내부 노드와 리프 노드로 구성된다.

B+Tree

- ├ 내부 노드: 탐색 경로 결정
- └ 리프 노드: 인덱스 키와 레코드 정보 저장

[내부 노드]

내부 노드는 다음 정보를 저장한다.

- 탐색 기준이 되는 키
- 자식 페이지를 가리키는 포인터

[자식 포인터 | 키 | 자식 포인터 | 키 | 자식 포인터]

예를 들어, 다음 내부 노드가 있다고 하자.

[30 | 60]

이 키들은 탐색 범위를 나누는 경계값이다.

30보다 작은 값

→ 첫 번째 자식 페이지

30 이상 60보다 작은 값

→ 두 번째 자식 페이지

60 이상인 값

→ 세 번째 자식 페이지

[30 | 60]
/ | \
< 30 30~59 ≥ 60

범위의 정확한 포함 규칙은 구현에 따라 다를 수 있지만, 내부 키가 자식 페이지의 범위를 구분한다는 원리는 같다.

[리프 노드]

리프 노드는 다음 정보를 저장한다.

- 인덱스 키
- 실제 레코드

또는

- 실제 레코드를 찾기 위한 위치 정보
 - 이전·다음 리프 페이지를 가리키는 연결 정보
-
- 인덱스 키: 실제 레코드를 빠르게 찾기 위한 검색 기준값
 - 실제 레코드: 테이블에 저장된 하나의 완전한 행 데이터

[10 | 레코드 정보] [20 | 레코드 정보]

↔

[30 | 레코드 정보] [40 | 레코드 정보]

인덱스가 어떤 방식으로 구성되었는지에 따라 리프 노드의 내용이 달라질 수 있다.

인덱스 형태	리프 노드에 저장될 수 있는 정보
클러스터형 인덱스	실제 행 데이터
비클러스터형·보조 인덱스	실제 행을 찾기 위한 기본 키 또는 레코드 식별자
일반적인 설명	데이터 페이지 번호와 슬롯 번호 등의 위치 정보

따라서 “리프 노드가 실제 레코드를 가리킨다”는 말은 항상 RAM 주소를 저장한다는 뜻이 아니다.

실제 데이터는 기본적으로 SSD/HDD의 데이터베이스 파일에 영구 저장되고, DBMS가 해당 데이터를 읽을 때 RAM의 버퍼 풀로 가져온다.

[2] [내부 노드와 리프 노드에 같은 키가 존재하는 이유]

B+Tree에서는 내부 노드에 있는 키가 리프 노드에도 다시 나타날 수 있다.

```

      [30 | 60]
      /  |  \
    [10 | 20] [30 | 40] [60 | 70]
  
```

여기서 30과 60은 내부 노드와 리프 노드에 모두 존재한다.

각 위치에서 역할이 다르기 때문이다.

내부 노드의 30

→ 어느 자식 페이지로 이동할지 결정하는 경계값

리프 노드의 30

→ 인덱스 키 30의 검색 결과와 레코드 정보를 제공

즉, 내부 노드의 키는 길을 안내하고 리프 노드의 키는 최종 검색 결과를 제공한다.

내부 키

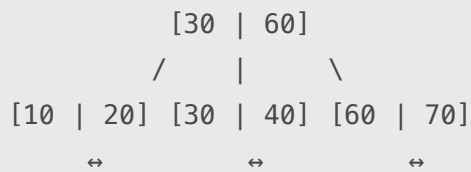
→ 안내판

리프 키

→ 실제 검색 항목

[3] [B+Tree의 탐색 과정]

다음 B+Tree에서 키 40을 찾는다고 해보자.



탐색 과정은 다음과 같다.

1. 루트 내부 노드에서 40과 30, 60을 비교한다.
2. $30 \leq 40 < 60$ 이므로 가운데 자식 페이지를 선택한다.
3. 가운데 리프 노드 [30 | 40]으로 이동한다.
4. 리프 노드에서 키 40을 찾는다.
5. 키 40과 연결된 실제 레코드 또는 레코드 위치 정보를 확인한다.

B+Tree는 내부 노드에서 같은 키를 발견하더라도 일반적으로 탐색을 끝내지 않고 리프 노드까지 내려간다.

내부 노드

→ 탐색 경로만 결정

리프 노드

→ 최종 검색 결과 확인

탐색 시간 복잡도는 다음과 같다.

$O(\log N)$

더 정확히는 노드 하나가 가질 수 있는 자식 수를 M이라고 할 때 트리 높이는 대략 다음과 같다.

$O(\log_m N)$

B+Tree는 M이기 때문에 실제 트리 높이가 매우 낮게 유지된다.

[4] [실제 레코드에 접근하는 과정]

인덱스에서 키를 찾았다고 해서 반드시 테이블의 전체 행을 바로 얻는 것은 아니다.
리프 노드가 무엇을 저장하는지는 인덱스 종류와 DBMS 구현에 따라 달라진다.

[리프 노드가 실제 행을 저장하는 경우]

B+Tree 탐색

↓

리프 노드 도착

↓

리프 노드에서 실제 행 데이터 확인

예를 들어, 클러스터형 인덱스 계열에서는 리프 노드가 실제 행 데이터를 포함할 수 있다.

[리프 노드가 행의 위치 정보만 저장하는 경우]

B+Tree 탐색

↓

리프 노드에서 레코드 위치 정보 확인

↓

해당 데이터 페이지 탐색

↓

실제 행 데이터 확인

예를 들어, 리프 노드는 다음과 같은 정보를 가질 수 있다.

인덱스 키: 40

레코드 위치: 데이터 페이지 120번, 슬롯 3번

개념적인 접근 과정은 다음과 같다.

B+Tree 리프 노드

[40 | 페이지 120, 슬롯 3]

↓

SSD/HDD의 데이터 페이지 120

↓

페이지 내부 슬롯 3

↓

실제 레코드

다만 실제 DBMS에서는 물리적인 페이지 주소 대신 기본 키나 DBMS가 관리하는
레코드 식별자를 저장할 수도 있다.

[5] [데이터베이스 페이지와 B+Tree]

데이터베이스의 테이블과 인덱스는 기본적으로 SSD/HDD에 페이지 단위로 저장된다.

B+Tree 노드 하나가 보통 데이터베이스의 인덱스 페이지 하나에 대응한다.

SSD/HDD의 인덱스 페이지

키	자식 페이지 번호	키	자식 페이지 번호
---	-----------	---	-----------

쿼리를 실행하면 DBMS는 필요한 인덱스 페이지를 RAM의 버퍼 풀로 가져온다.

SSD/HDD의 B+Tree 페이지

↓ 페이지 I/O

RAM의 Buffer Pool

↓

CPU가 키 비교

↓

다음 자식 페이지 결정

이미 필요한 페이지가 버퍼 풀에 있다면 저장장치에서 다시 읽지 않고 RAM에서 바로 사용한다.

필요한 페이지 확인

↓

RAM에 있음 → 바로 탐색

RAM에 없음 → SSD/HDD에서 읽어옴

B+Tree는 노드 하나에 여러 키를 저장하므로 한 번의 페이지 I/O로 많은 키를 확인할 수 있다.

[6] [B+Tree의 Fan-out]

Fan-out은 노드 하나가 가질 수 있는 자식 수를 의미한다.

이진 트리

→ 자식 최대 2개

B+Tree

→ 자식 수십~수백 개 이상 가능

B+Tree 내부 노드는 주로 키와 자식 포인터만 저장하므로 같은 크기의 페이지에 많은 항목을 저장할 수 있다.

내부 노드에 실제 행 데이터를 저장하지 않음



한 페이지에 더 많은 키와 포인터 저장



Fan-out 증가



트리 높이 감소



페이지 I/O 감소

예를 들어, 한 노드가 자식 500개를 가질 수 있다고 단순하게 가정하면 다음과 같다.

높이 1 → 약 500개의 범위

높이 2 → 약 250,000개의 범위

높이 3 → 약 125,000,000개의 범위

실제로는 페이지 크기, 키 크기, 포인터 크기, 페이지 사용률에 따라 달라지지만 작은 높이로 매우 많은 데이터를 관리할 수 있다는 원리는 같다.

[7] [B+Tree의 범위 검색]

B+Tree의 가장 큰 장점 중 하나는 범위 검색이다.

리프 노드는 키 순서대로 정렬되어 있으며 서로 연결되어 있다.

[10 | 20] ↔ [30 | 40] ↔ [50 | 60] ↔ [70 | 80]

다음 쿼리를 실행한다고 해보자.

```
SELECT *  
FROM users  
WHERE age BETWEEN 20 AND 60;
```

B+Tree는 먼저 20이 저장된 리프 노드를 탐색한다.

그 후 리프 노드의 연결을 따라 60까지 순서대로 읽는다. 이 때문에 다음과 같은 연산에 유리하다.

```
WHERE age BETWEEN 20 AND 30;
```

```
WHERE age >= 20;
```

```
ORDER BY age;
```

```
SELECT MIN(age);
```

```
SELECT MAX(age);
```


[8] [B+Tree의 삽입]

B+Tree는 새로운 키를 정렬된 위치의 리프 노드에 삽입한다.
다음 리프 노드가 최대 3개의 키만 저장할 수 있다고 가정하자.

[10 | 20 | 30]

여기에 40을 삽입하면 노드가 저장할 수 있는 최대 키 개수를 초과한다.

[10 | 20 | 30 | 40] ← Overflow

이 경우 리프 노드를 둘로 분할한다.

[10 | 20] ↔ [30 | 40]

새로 만들어진 오른쪽 리프노드의 첫 번째 키를 부모 내부 노드에 복사한다.

 [30]
 / \
[10 | 20] ↔ [30 | 40]

여기서 30은 부모 내부 노드와 리프 노드에 모두 남아 있다 .

부모의 30

→ 탐색 경계값

리프의 30

→ 실제 인덱스 항목

부모 노드도 가득 차면 부모 노드 역시 분할된다.

리프 노드 분할

↓

부모에 경계 키 추가

↓

부모도 가득 참

↓

부모 노드 분할

↓

필요하면 새로운 루트 생성

루트가 분할될 때 트리 높이가 1증가한다.

[9] [B+Tree의 삭제]

키는 리프 노드에서 삭제한다.

삭제 후 리프 노드의 키가 최소 개수보다 적어지면 균형을 복구해야 한다.

리프에서 키 삭제

↓

최소 키 개수 미달

↓

형제 노드에서 키를 빌림

또는

형제 노드와 병합

[형제 노드에서 키를 빌리는 경우]

형제 노드에 여유가 있으면 키를 하나 가져와 재분배한다.

재분배 전

[10] ↔ [20 | 30 | 40]

재분배 후

[10 | 20] ↔ [30 | 40]

리프 노드의 첫 번째 키가 바뀌면 부모 내부 노드의 경계 키도 함께 갱신해야 할 수 있다.

[형제 노드와 병합하는 경우]

형제에게 빌릴 키가 없다면 두 노드를 하나로 합친다.

병합 전

[10] ↔ [20]

병합 후

[10 | 20]

병합으로 자식 페이지가 하나 줄어들면 부모 노드의 키와 자식 포인터도 제거된다.

부모 노드까지 최소 키 개수보다 작아지면 위쪽에서도 재분배나 병합이 발생할 수 있다.

[10] [B-Tree와 B+Tree의 차이]

구분	B-Tree	B+Tree
내부 노드	데이터 정보 저장 가능	주로 탐색용 키와 자식 포인터
리프 노드	일부 키와 데이터 저장	모든 인덱스 키와 데이터 정보 저장
탐색 종료	내부 노드에서 끝날 수 있음	일반적으로 리프에서 끝남
키 중복	보통 각 키가 한 위치에 존재	내부 경계 키가 리프에도 존재 가능
리프 연결	필수 아님	순서대로 연결
범위 검색	가능	더 효율적
Fan-out	상대적으로 작을 수 있음	내부 노드가 단순하여 상대적으로 큼
주요 사용	일부 파일 시스템과 저장 구조	데이터베이스 인덱스에서 널리 사용

[11] [Hash Index와 B+Tree의 차이]

Hash Index는 정확히 일치하는 값을 찾는 데 유리하다.

```
WHERE id = 100;
```

하지만 해시 함수는 키의 정렬 순서를 유지하지 않는다.

따라서 다음과 같은 범위 검색에는 적합하지 않다.

```
WHERE age BETWEEN 20 AND 30;
```

```
WHERE age > 20;
```

```
ORDER BY age;
```

B+Tree는 인덱스 키를 정렬된 상태로 유지하고 리프 노드끼리 연결한다.

검색 조건	Hash Index	B+Tree Index
id = 100	평균적으로 매우 빠름	빠름
age BETWEEN 20 AND 30	부적합	효율적
age > 20	부적합	효율적
ORDER BY age	정렬 작업 필요	인덱스 순서 활용 가능
MIN, MAX	불리함	양 끝 리프 활용 가능
순차 탐색	불리함	리프 연결 활용 가능